

"""

verrouillage_1_4_heisenberg_commutateur.py
=====

Objectif : verrouiller le point 1.4 de l'audit V107+1.

Définir précisément les opérateurs position $\hat{X}$ et impulsion $\hat{P}$ sur $\ell^2(\Omega_N)$,
calculer leur commutateur, et montrer qu'il est NON NUL.

Définitions formelles (V107+1) :

$\hat{X}$ = opérateur de "position simpliciale" :

$$(\hat{X} \psi)(C) := n_{\text{tri}}(C) \cdot \psi(C)$$

$\hat{P}$ = générateur infinitésimal du semi-groupe MCMC, transcription discrète
de l'opérateur dérivée :

$$(\hat{P} \psi)(C) := -i \cdot \sum_{C' \sim C} k(C, C') \cdot (\psi(C') - \psi(C))$$

où $k(C, C')$ = taux de transition Metropolis = $\min(1, \exp((S(C') - S(C))/\tau_{\text{eff}}))$
et $C' \sim C$ signifie " C' obtenu de C par un swap valide"

Théorème (commutateur).

$$([\hat{X}, \hat{P}] \psi)(C) = -i \cdot \sum_{C' \sim C} k(C, C') \cdot [n_{\text{tri}}(C) - n_{\text{tri}}(C')] \cdot \psi(C')$$

Preuve :

$$(\hat{P} \hat{X} \psi)(C) = -i \cdot \sum_{C'} k(C, C') \cdot (n_{\text{tri}}(C') \psi(C') - n_{\text{tri}}(C) \psi(C))$$

$$(\hat{X} \hat{P} \psi)(C) = n_{\text{tri}}(C) \cdot (\hat{P} \psi)(C) = -i \cdot n_{\text{tri}}(C) \cdot \sum_{C'} k(C, C') \cdot (\psi(C') - \psi(C))$$

$$[\hat{X}, \hat{P}] \psi = (\hat{X} \hat{P} - \hat{P} \hat{X}) \psi$$

$$= -i \cdot n_{\text{tri}}(C) \cdot \sum_{C'} k(C, C') \cdot (\psi(C') - \psi(C))$$

$$- (-i) \cdot \sum_{C'} k(C, C') \cdot (n_{\text{tri}}(C') \psi(C') - n_{\text{tri}}(C) \psi(C))$$

$$= -i \cdot \sum_{C'} k(C, C') \cdot [n_{\text{tri}}(C) \psi(C') - n_{\text{tri}}(C) \psi(C) - n_{\text{tri}}(C') \psi(C') +$$

$$= -i \cdot \sum_{C'} k(C, C') \cdot [n_{\text{tri}}(C) - n_{\text{tri}}(C')] \cdot \psi(C')$$

■

Conséquence immédiate.

$[\hat{X}, \hat{P}] = M \neq 0$ dès qu'il existe au moins une transition $C \rightarrow C'$ avec
 $k(C, C') > 0$ et $n_{\text{tri}}(C) \neq n_{\text{tri}}(C')$.

Ce qui est trivialement vrai dans le noyau edge-swap canonique : la plupart
des swaps changent n_{tri} (création/destruction de triangles).

Inégalité de Heisenberg combinatoire (forme robuste).

Pour tout état ψ normalisé, par l'inégalité de Robertson :

$$\sigma(\hat{X}) \cdot \sigma(\hat{P}) \geq \frac{1}{2} |\langle [\hat{X}, \hat{P}] \rangle_{\psi}|$$

Le second membre est mesurable empiriquement. Pour π_N (mesure de Gibbs) :

$$\{[\hat{X}, \hat{P}]\}_{\pi_N} = -i \cdot E_{\pi_N}[\sum_{C'} k(C, C') \cdot (n_{\text{tri}}(C) - n_{\text{tri}}(C'))]$$

Cette quantité est non nulle pour la dynamique canonique.

VÉRIFICATION NUMÉRIQUE (chemin 3 Rasoir d'Ockham) :

- Sampling MCMC à l'équilibre
- Pour chaque config C visitée, énumérer un échantillon de voisins C'
- Mesurer empiriquement $E[k(C, C') \cdot (n_{\text{tri}}(C) - n_{\text{tri}}(C'))]$
- Comparer à la variance de n_{tri} ($\sigma(\hat{X})^2$)

"""

```
import numpy as np
import math
import time
from collections import deque
from scipy.spatial import KDTree

KAPPA_0 = 8
BETA = 0.30
TAU = 0.50
N_TEST = 150
N_SEEDS = 3
N_SAMPLES_EQUILIBRIUM = 100 # configurations échantillonnées en équilibre
N_NEIGHBORS_PER_CONFIG = 30 # voisins testés par configuration

# Pipeline minimal
def _rips_radius(N, kappa=KAPPA_0): return (kappa / ((N-1) * (4/3) * math.pi)) **
def _E0(N, kappa=KAPPA_0): return int(N * kappa / 2)

def _lcc_size(adj, N):
    visited = [False]*N; best = 0
    for s in range(N):
        if not visited[s]:
            sz = 0; q = deque([s]); visited[s] = True
            while q:
                v = q.popleft(); sz += 1
                for w in adj[v]:
                    if not visited[w]: visited[w] = True; q.append(w)
            best = max(best, sz)
    return best

def _build_initial(N, seed):
    r = _rips_radius(N); r_p = 1.5 * r
    E0 = _E0(N)
    rng = np.random.default_rng(seed)
```

```

pts = rng.uniform(0.0, 1.0, (N, 3))
tree = KDTree(pts)
adj = [set() for _ in range(N)]
for i, j in tree.query_pairs(r):
    adj[i].add(j); adj[j].add(i)
pool = list(tree.query_pairs(r_p))
rng2 = np.random.default_rng(seed + 50_000)
K = sum(len(adj[i]) for i in range(N)) // 2
att = 0; max_att = 10 * abs(E0 - K) + 5000
while K != E0 and att < max_att:
    att += 1
    if K < E0:
        e = pool[rng2.integers(len(pool))]; u, v = e
        if v not in adj[u]: adj[u].add(v); adj[v].add(u); K += 1
    else:
        edges = [(i, j) for i in range(N) for j in adj[i] if j > i]
        if not edges: break
        e = edges[rng2.integers(len(edges))]; u, v = e
        adj[u].discard(v); adj[v].discard(u)
        if _lcc_size(adj, N) == N: K -= 1
        else: adj[u].add(v); adj[v].add(u)
return pts, adj, pool, E0

def _count_triangles(adj, N):
    n = 0
    for i in range(N):
        for j in adj[i]:
            if j > i:
                n += len([k for k in adj[i] & adj[j] if k > j])
    return n

def compute_S(adj, n_tri, N):
    degs = np.array([len(adj[i]) for i in range(N)], float)
    D = degs.std() / (degs.mean() + 1e-8)
    return math.log(1 + n_tri) - BETA * D

def _fast_still_connected(adj, u, v, N):
    visited = {u}; q = deque([u])
    while q:
        node = q.popleft()
        for w in adj[node]:
            if w in visited: continue
            if node == u and w == v: continue
            if node == v and w == u: continue
            visited.add(w); q.append(w)
            if w == v: return True
    return False

```

```

def run_mcmc_with_samples(pts, adj_init, pool, N, E0, seed, n_samples=N_SAMPLES_E
    """MCMC qui sample n_samples configs en équilibre."""
    tau_eff = TAU / N
    rng = np.random.default_rng(seed + 200_000)
    adj = [set(a) for a in adj_init]
    edge_list = [(i, j) for i in range(N) for j in adj[i] if j > i]
    n_tri = _count_triangles(adj, N)
    S_curr = compute_S(adj, n_tri, N)
    burn = 100 * N
    interval = max(1, 2 * N)
    n_steps = burn + n_samples * interval
    pool_arr = np.array(pool, dtype=np.int32)
    n_pool = len(pool_arr)
    samples = []
    n_tri_history = []

    for step in range(n_steps):
        if not edge_list: break
        idx_out = int(rng.integers(len(edge_list)))
        u, v = edge_list[idx_out]
        e_in = None
        for _ in range(30):
            idx_in = int(rng.integers(n_pool))
            a, b = int(pool_arr[idx_in, 0]), int(pool_arr[idx_in, 1])
            if b not in adj[a]: e_in = (a, b); break
        if e_in is None: continue
        a, b = e_in
        if len(adj[u]) <= 2 or len(adj[v]) <= 2:
            if not _fast_still_connected(adj, u, v, N): continue
        tri_lost = len(adj[u] & adj[v])
        adj[u].discard(v); adj[v].discard(u)
        adj[a].add(b); adj[b].add(a)
        tri_gained = len(adj[a] & adj[b])
        n_tri_new = n_tri - tri_lost + tri_gained
        S_new = compute_S(adj, n_tri_new, N)
        dS = S_new - S_curr
        if dS >= 0 or rng.random() < math.exp(dS / tau_eff):
            n_tri = n_tri_new; S_curr = S_new
            edge_list[idx_out] = (a, b)
        else:
            adj[u].add(v); adj[v].add(u); adj[a].discard(b); adj[b].discard(a)
        if step >= burn and (step - burn) % interval == 0:
            samples.append([set(s) for s in adj], n_tri, S_curr)
            n_tri_history.append(n_tri)

    return samples, np.array(n_tri_history)

```

```

def sample_neighbors_and_compute(adj, n_tri_C, S_C, pool, N, n_neighbors=N_NEIGHB
"""
Sample n_neighbors voisins de C, calcule pour chacun :
 $k(C, C') \cdot (n_{\text{tri}}(C) - n_{\text{tri}}(C'))$ 
et retourne la liste.
"""

rng = np.random.default_rng(seed)
tau_eff = TAU / N
edges = [(i, j) for i in range(N) for j in adj[i] if j > i]
pool_arr = np.array(pool, dtype=np.int32)

contributions = []
diffs_ntri = []
tries = 0
max_tries = n_neighbors * 5

while len(contributions) < n_neighbors and tries < max_tries:
    tries += 1
    e_out = edges[rng.integers(len(edges))]
    u, v = e_out
    idx_in = int(rng.integers(len(pool_arr)))
    a, b = int(pool_arr[idx_in, 0]), int(pool_arr[idx_in, 1])
    if b in adj[a]: continue
    if e_out == (a, b): continue

    # Test connexité
    if len(adj[u]) <= 2 or len(adj[v]) <= 2:
        adj_t = [set(s) for s in adj]
        adj_t[u].discard(v); adj_t[v].discard(u)
        if not _fast_still_connected(adj_t, u, v, N):
            continue

    # Calcul n_tri(C')
    tri_lost = len(adj[u] & adj[v])
    common_after = (adj[a] | {b}) & (adj[b] | {a}) # ne pas appliquer le swa
    # Plus simple : simulons le swap, comptons, défaisons
    adj[u].discard(v); adj[v].discard(u)
    adj[a].add(b); adj[b].add(a)
    tri_gained = len(adj[a] & adj[b])
    n_tri_Cprime = n_tri_C - tri_lost + tri_gained

    # Calcul S(C')
    S_Cprime = compute_S(adj, n_tri_Cprime, N)

    # Défaire

```

```

adj[u].add(v); adj[v].add(u)
adj[a].discard(b); adj[b].discard(a)

# Taux Metropolis :  $k(C, C') = \min(1, \exp((S(C') - S(C)) / \tau_{\text{eff}}))$ 
dS = S_Cprime - S_C
k_CC = min(1.0, math.exp(dS / tau_eff)) if dS < 0 else 1.0

# Contribution au commutateur :  $k \cdot (n_{\text{tri}}(C) - n_{\text{tri}}(C'))$ 
delta_ntri = n_tri_C - n_tri_Cprime
contributions.append(k_CC * delta_ntri)
diffs_ntri.append(delta_ntri)

return contributions, diffs_ntri

if __name__ == "__main__":
    print("=" * 70)
    print("VERROUILLAGE 1.4 – Commutateur Heisenberg combinatoire  $[\hat{X}, \hat{P}]$ ")
    print("=" * 70)
    print(f"\nN = {N_TEST}, paramètres canoniques  $\kappa_0=\{\text{KAPPA}_0\}$ ,  $\beta=\{\text{BETA}\}$ ,  $\tau=\{\text{TAU}\}$ ")
    print(f"\nObjectif : vérifier que  $[\hat{X}, \hat{P}] \neq 0$  et estimer  $\langle [\hat{X}, \hat{P}] \rangle_{\{\pi_N\}}$ ")

    print("\n[Théorème – preuve analytique fournie en docstring]")
    print("  $([\hat{X}, \hat{P}] \psi)(C) = -i \cdot \sum_{C' \sim C} k(C, C') \cdot [n_{\text{tri}}(C) - n_{\text{tri}}(C')] \cdot \psi(C)$ ")
    print("\n[Vérification numérique sur l'équilibre  $\pi_N$ ]")

    all_imag_parts_per_seed = []
    sigma_ntri_per_seed = []

    for seed in range(N_SEEDS):
        t0 = time.time()
        pts, adj_init, pool, E0 = _build_initial(N_TEST, seed)
        if _lcc_size(adj_init, N_TEST) / N_TEST < 0.9: continue

        # Sampling équilibre
        samples, n_tri_hist = run_mcmc_with_samples(pts, adj_init, pool, N_TEST,
        t_mcmc = time.time() - t0

        if len(samples) < 30: continue

        #  $\sigma(\hat{X})^2 = \text{Var}(n_{\text{tri}})$  en équilibre
        sigma_X_sq = float(n_tri_hist.var())
        sigma_X = math.sqrt(sigma_X_sq)
        sigma_ntri_per_seed.append(sigma_X)

        # Pour chaque config échantillonnée, échantillonner des voisins
        # et calculer la partie imaginaire de  $\langle [\hat{X}, \hat{P}] \rangle$ 

```

```

imag_per_config = []
for i, (adj_s, n_tri_s, S_s) in enumerate(samples):
    contribs, diffs = sample_neighbors_and_compute(
        adj_s, n_tri_s, S_s, pool, N_TEST, seed=seed * 1000 + i
    )
    if contribs:
        #  $\{[\hat{X}, \hat{P}]\}$  a valeur imaginaire pure ; on collecte les coefficients
        imag_per_config.append(np.mean(contribs))

if imag_per_config:
    imag_mean = float(np.mean(imag_per_config))
    imag_std = float(np.std(imag_per_config))
    all_imag_parts_per_seed.append(imag_mean)

    print(f"\nSeed {seed} (t={t_mcmc:.1f}s) :")
    print(f"  $\sigma(\hat{X}) = \sigma(n\_tri)$  en équilibre = {sigma_X:.2f}")
    print(f"  $\{[\hat{X}, \hat{P}]\} / (-i) \approx \{imag\_mean:.4f\} \pm \{imag\_std:.4f\}$ ")
    print(f"  $\rightarrow \frac{1}{2}|\{[\hat{X}, \hat{P}]\}| \approx \{abs(imag\_mean)/2:.4f\}$ ")
    print(f" Borne Robertson :  $\sigma(\hat{X}) \cdot \sigma(\hat{P}) \geq \{abs(imag\_mean)/2:.4f\}$ ")
    print(f"  $\Rightarrow \sigma(\hat{P}) \geq \{abs(imag\_mean)/(2*sigma\_X):.6f\}$ ")

if all_imag_parts_per_seed:
    print(f"\n{'='*70}")
    print("RÉSULTAT AGRÉGÉ (3 seeds)")
    print(f"{'='*70}")
    arr = np.array(all_imag_parts_per_seed)
    sig = np.array(sigma_ntri_per_seed)
    print(f"  $\{[\hat{X}, \hat{P}]\} / (-i)$  : moyenne = {arr.mean():.4f}, std = {arr.std():.4f}")
    print(f"  $\sigma(\hat{X})$  : moyenne = {sig.mean():.2f}, std = {sig.std():.2f}")

    if abs(arr.mean()) > 0.01:
        print(f"\n ✓  $[\hat{X}, \hat{P}] \neq 0$  confirmé numériquement.")
        print(f" ✓ Inégalité de Heisenberg combinatoire NON TRIVIALE :")
        print(f"  $\sigma(\hat{X}) \cdot \sigma(\hat{P}) \geq \{abs(arr.mean())/2:.4f\}$ ")
        print(f"\n Verdict : T12.3  $[T^*] \rightarrow [T]$  partiel.")
        print(f" Le calcul analytique du commutateur est rigoureux ;")
        print(f" la vérification numérique confirme une partie imaginaire non nulle")
    else:
        print(f"\n  $\Delta$  Partie imaginaire mesurée petite, à investiguer.")

print(f"\n{'='*70}")
print("Note méthodologique :")
print(f"{'='*70}")
print(" Le commutateur  $[\hat{X}, \hat{P}]$  est analytiquement non nul (preuve par calcul)")
print(" La vérification numérique mesure l'EFFET de ce commutateur sur la mesure")
print(" de Gibbs  $\pi_N$  (à travers la valeur moyenne  $\{[\hat{X}, \hat{P}]\}_{\pi_N}$ ).")
print(" La valeur précise dépend de N et de la dynamique exacte ;")

```

```
print(" ce qui importe est que cette valeur soit non nulle, ce qui suffit à"  
print(" établir une inégalité de Heisenberg combinatoire NON TRIVIALE.")
```